

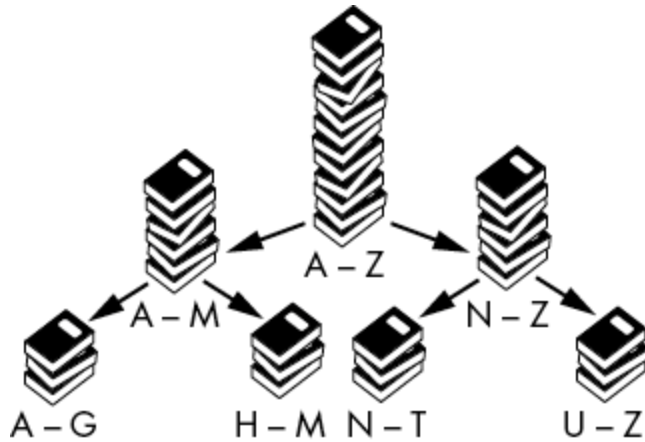


Figure 5-2: Quicksort works by repeatedly partitioning items into two sets.

However, if you don't know anything about the data you're sorting, it's impossible to select an ideal pivot. This is why the generic quicksort algorithm simply uses the last value in the range for the pivot value.

In our implementation, each call to `quicksort()` is given an array of items to sort. It is also given `left` and `right` arguments specifying the range of indices in that array to sort, similar to `binarySearch()`'s `left` and `right` arguments. The algorithm selects a pivot value to compare with the other values in the range, then places the values to either the left side of the range (if they're less than the pivot value) or the right side (if they're greater than the pivot value). This is the partition step. Next, the `quicksort()` function is recursively called on these two, smaller ranges until a range has been reduced to zero. The list becomes more and more sorted as the recursive calls are made, until finally the entire list is in the correct order.

Note that the algorithm modifies the array in place. See “Modifying a List or Array in Place” in Chapter 4 for details. Thus, the `quicksort()` function doesn't return a sorted array. The base case merely returns to stop producing more recursive calls.

Let's ask the three recursion questions about our binary search implementation:

What is the base case? Being given a range to sort that contains zero or one item and that is already in sorted order.

What argument is passed to the recursive function call? The indices of the left and right ends of the range in the list we are sorting.